

1. Edit config/server.properties

Set:

process.roles=broker,controller

node.id=1

listeners=PLAINTEXT://localhost:9092,CONTROLLER://localhost:9093

advertised.listeners=PLAINTEXT://localhost:9092

listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

inter.broker.listener.name=PLAINTEXT

controller.listener.names=CONTROLLER

controller.quorum.bootstrap.servers=localhost:9093

log.dirs=D:/NIC_Kanpur/kafka/data/kafka-logs

2. Generate Cluster ID

bin\windows\kafka-storage.bat random-uuid

Example:

abcd1234-....

3. Format

bin\windows\kafka-storage.bat format ^

--standalone ^

--cluster-id YOUR_CLUSTER_ID ^

--config config\server.properties

This is the officially documented standalone format for local development.

4. Start Kafka

bin\windows\kafka-server-start.bat config\server.properties

5. Create Topic

bin\windows\kafka-topics.bat --create ^

--topic order-created^

--bootstrap-server localhost:9092

6. Start Consumer

```
bin\windows\kafka-console-consumer.bat ^
```

```
--bootstrap-server localhost:9092 ^
```

```
--topic order-created ^
```

```
--from-beginning
```

After Kafka is running, **keep that Command Prompt open**. Do **not** use it for any other commands.

For Day 3, you'll typically have **4 Command Prompt windows**.

Command Prompt 1 – Kafka Server

Start Kafka and leave this window running.

```
cd D:\NIC_Kanpur\kafka
```

```
bin\windows\kafka-server-start.bat config\server.properties
```

You should see logs continuously. **Do not close this window.**

Command Prompt 2 – Create Topic

Open a **new** Command Prompt.

```
cd D:\NIC_Kanpur\kafka
```

```
bin\windows\kafka-topics.bat --create ^
```

```
--topic order-events ^
```

```
--bootstrap-server localhost:9092 ^
```

```
--partitions 1 ^
```

```
--replication-factor 1
```

Verify:

```
bin\windows\kafka-topics.bat --list ^
```

```
--bootstrap-server localhost:9092
```

Expected:

```
order-events
```

Command Prompt 3 – Kafka Consumer

Open another Command Prompt.

```
cd D:\NIC_Kanpur\kafka
```

```
bin\windows\kafka-console-consumer.bat ^
```

```
--bootstrap-server localhost:9092 ^
```

```
--topic order-created ^
```

```
--from-beginning
```

Keep this window open. When your Order Service publishes an event, you'll see the JSON message here.

Command Prompt 4 – Kafka Producer (Optional)

You can test Kafka without Spring Boot by opening another Command Prompt.

```
cd D:\NIC_Kanpur\kafka
```

```
bin\windows\kafka-console-producer.bat ^
```

```
--bootstrap-server localhost:9092 ^
```

```
--topic order-events
```

Type:

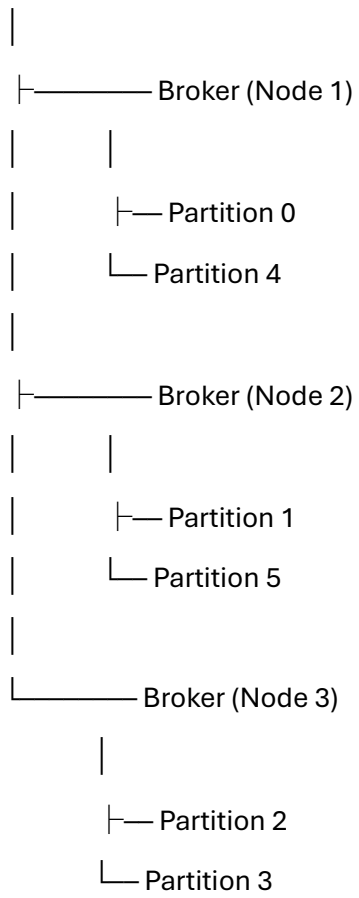
Hello Kafka

Press **Enter**.

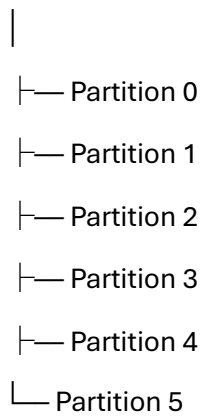
The consumer window should immediately display:

Hello Kafka

Kafka Cluster



Topic



Once Kafka is working, the sequence should be:

1. Start Config Server
2. Start Eureka Server
3. Start Product Service
4. Start Customer Service
5. Start Kafka Server
6. Create order-created topic (one time)
7. Start Kafka Consumer
8. Start Order Service
9. Start API Gateway
10. Place an order using Postman
11. Observe the event in the Kafka Consumer window

Postman Examples

Create Order

POST

http://localhost:9090/api/v1/orders

Body

```
{  
  "customerId": 1,  
  "productId": 1,  
  "quantity": 2  
}
```

Get All Orders

GET http://localhost:9090/api/v1/orders

Get Order By Id

GET http://localhost:9090/api/v1/orders/1

Update Status

PUT <http://localhost:9090/api/v1/orders/1/status/SHIPPED>

Possible values:

- CREATED
 - CONFIRMED
 - PROCESSING
 - SHIPPED
 - DELIVERED
 - CANCELLED
-

Delete Order

DELETE <http://localhost:9090/api/v1/orders/1>

Project Progress

 **Postman Examples**

Create Order

POST

<http://localhost:9090/api/v1/orders>

Body

```
{  
  "customerId": 1,  
  "productId": 1,  
  "quantity": 2  
}
```

Get All Orders

GET <http://localhost:9090/api/v1/orders>

Get Order By Id

GET <http://localhost:9090/api/v1/orders/1>

Update Status

PUT http://localhost:9090/api/v1/orders/1/status/SHIPPED

Possible values:

- CREATED
- CONFIRMED
- PROCESSING
- SHIPPED
- DELIVERED
- CANCELLED

Delete Order

DELETE http://localhost:9090/api/v1/orders/1

Apache Kafka Producer–Consumer: Complete End-to-End Flow

Apache Kafka is an **event-streaming platform** used to move data reliably between independent applications.

In our e-commerce microservices example:

Order Service

|
| publishes OrderCreatedEvent
v

Kafka Topic: order-created

|
+-----+
| |
v v

Inventory Service Notification Service

updates stock sends notification

The Order Service is the **producer**. Kafka stores the event in a **topic**. Inventory Service and Notification Service are **consumers**. Producers and consumers are independent: a producer does not directly call or wait for a consumer.

1. Important Kafka Terminology

1.1 Event or Message

An event represents something that happened in the system.

Examples:

OrderCreated

PaymentCompleted

InventoryUpdated

CustomerRegistered

A Kafka event is commonly represented as:

Key + Value + Headers + Timestamp

Example:

Key = ORD-1001

Value = OrderEvent object

Timestamp = 2026-07-29T20:30:00

Headers = content type, Java type, correlation ID, etc.

1.2 Producer

A producer is an application that publishes events to Kafka.

In your application:

Order Service = Kafka Producer

The producer normally uses:

KafkaTemplate<String, OrderEvent>

Example:

```
kafkaTemplate.send(  
    "order-created",  
    order.getOrderNumber(),  
    orderEvent  
);
```

Here:

Topic = order-created

Key = order number

Value = OrderEvent

1.3 Consumer

A consumer is an application that reads events from Kafka.

Example:

```
@KafkaListener(  
    topics = "${app.kafka.topic.order-created}",  
    groupId = "${spring.kafka.consumer.group-id}"  
)  
  
public void consumeOrderCreatedEvent(OrderEvent event) {  
    System.out.println(event);  
}
```

In your project:

Inventory Service = Consumer

Notification Service = Consumer

1.4 Broker

A Kafka broker is a running Kafka server.

Its responsibilities include:

- receiving records from producers;
- storing records;
- serving records to consumers;
- managing topic partitions;
- replicating partitions;
- maintaining offsets and consumer-group information.

For local development:

localhost:9092

means:

localhost = machine where Kafka is running

9092 = Kafka broker listener port

A Kafka cluster normally contains multiple brokers:

Broker 1: kafka1:9092

Broker 2: kafka2:9092

Broker 3: kafka3:9092

Kafka clients initially connect to the configured bootstrap servers and then discover the remaining brokers and topic-partition metadata.

1.5 Kafka Cluster

A Kafka cluster is a group of Kafka brokers working together.

Kafka Cluster

├— Broker 1

├— Broker 2

└— Broker 3

A multi-broker cluster provides:

- scalability;
- fault tolerance;
- partition distribution;
- replication;
- high availability.

1.6 Topic

A topic is a logical category in which related events are stored.

Examples:

order-created

payment-completed

inventory-updated

notification-requested

A topic is similar to an event stream, not exactly a traditional queue.

Example:

Topic: order-created

Event 0: Order 1001

Event 1: Order 1002

Event 2: Order 1003

1.7 Partition

Every topic is divided into one or more partitions.

order-created

└─ Partition 0

└─ Partition 1

└─ Partition 2

Each partition is an ordered, append-only log.

Example:

Partition 0

Offset 0 → Order 1001

Offset 1 → Order 1004

Offset 2 → Order 1007

Partition 1

Offset 0 → Order 1002

Offset 1 → Order 1005

Offset 2 → Order 1008

Kafka guarantees ordering **within a partition**, not automatically across the complete topic.

Partitions provide:

- parallel processing;
- scalability;
- higher throughput;
- distribution across brokers.

1.8 Record Key

The key determines which partition receives a record.

Example:

```
kafkaTemplate.send(  
    "order-created",  
    orderNumber,  
    orderEvent  
);
```

Here:

Key = orderNumber

Kafka normally calculates:

$$\text{partition} = \text{hash}(\text{key}) \% \text{numberOfPartitions}$$

Therefore, events with the same key generally go to the same partition.

For example:

Key ORD-1001 → Partition 1

Key ORD-1001 → Partition 1

Key ORD-1001 → Partition 1

This preserves ordering for that key.

Good key choices include:

orderId

customerId

productId

paymentId

The choice depends on the ordering requirement.

1.9 Record Value

The value contains the actual event data.

Example:

```
public class OrderEvent {  
  
    private Long orderId;  
    private String orderNumber;  
    private Long customerId;  
    private Long productId;  
    private Integer quantity;  
    private BigDecimal totalAmount;  
}
```

The value must be converted into bytes before Kafka can store it.

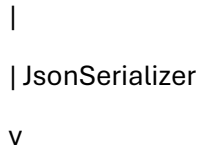
1.10 Serializer

A serializer converts a Java object into bytes.

Java object → Serializer → byte[]

Example:

OrderEvent object



JSON bytes

The producer uses serializers.

`spring.kafka.producer.key-serializer=\`

`org.apache.kafka.common.serialization.StringSerializer`

`spring.kafka.producer.value-serializer=\`

`org.springframework.kafka.support.serializer.JsonSerializer`

Kafka defines serializer abstractions for keys and values, while Spring Kafka provides JSON serialization support for Java objects.

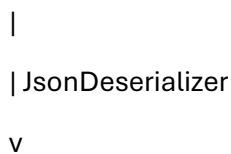
1.11 Deserializer

A deserializer performs the reverse conversion.

byte[] → Deserializer → Java object

Example:

JSON bytes



OrderEvent object

The consumer uses deserializers.

`spring.kafka.consumer.key-deserializer=\`

`org.apache.kafka.common.serialization.StringDeserializer`

`spring.kafka.consumer.value-deserializer=\`

`org.springframework.kafka.support.serializer.JsonDeserializer`

1.12 Offset

An offset is the sequential position of a record inside a partition.

Partition 0

Offset 0 → Event A

Offset 1 → Event B

Offset 2 → Event C

Offset 3 → Event D

Offsets start independently for each partition.

Therefore:

Partition 0 can have offset 5

Partition 1 can also have offset 5

A record is uniquely located using:

Topic + Partition + Offset

Example:

order-created, partition 1, offset 25

1.13 Consumer Group

A consumer group is a set of consumers that cooperate to process a topic.

`spring.kafka.consumer.group-id=inventory-group`

Consumers having the same `group.id` belong to the same consumer group.

Within one consumer group, a partition is assigned to only one consumer at a time.

Example:

Topic: order-created

Partitions: 3

Consumer group: inventory-group

Consumer 1 → Partition 0

Consumer 2 → Partition 1

Consumer 3 → Partition 2

This allows parallel processing.

1.14 Different Consumer Groups

Different groups independently receive the same events.

Topic: order-created

```
|  
+--> inventory-group  
  
|  
+--> notification-group  
  
|  
+--> analytics-group
```

Each group maintains its own offsets.

Therefore:

Inventory Service gets every order event.

Notification Service also gets every order event.

Analytics Service also gets every order event.

Example configuration:

Inventory Service

spring.kafka.consumer.group-id=inventory-group

Notification Service

spring.kafka.consumer.group-id=notification-group

They must use different group IDs because they perform different work.

1.15 Consumer Instance

A consumer instance is one running consumer application or listener container.

Suppose Inventory Service has three instances:

Inventory Service instance 1

Inventory Service instance 2

Inventory Service instance 3

If all use:

spring.kafka.consumer.group-id=inventory-group

Kafka distributes partitions among them.

1.16 Rebalancing

Rebalancing occurs when Kafka redistributes partitions among consumers in a group.

It can happen when:

- a consumer starts;
- a consumer stops;
- a consumer crashes;
- the number of partitions changes;
- group membership changes;
- a consumer becomes unresponsive.

Before:

Consumer 1 → Partition 0, Partition 1

Consumer 2 → Partition 2

After Consumer 3 joins:

Consumer 1 → Partition 0

Consumer 2 → Partition 1

Consumer 3 → Partition 2

During rebalancing, processing may pause briefly.

1.17 Leader and Replica

Each partition has one leader and possibly multiple replicas.

Partition 0

Broker 1 → Leader

Broker 2 → Follower replica

Broker 3 → Follower replica

Producer and consumer requests normally go through the partition leader.

Followers copy data from the leader.

If the leader broker fails, Kafka can elect an eligible replica as the new leader.

1.18 Replication Factor

The replication factor specifies how many copies of each partition Kafka maintains.

Replication factor = 3

means:

1 leader copy + 2 follower copies

For local single-broker development:

Replication factor = 1

For production, a replication factor of three is common, provided the cluster has at least three brokers.

1.19 Retention

Kafka does not normally delete an event immediately after a consumer reads it.

It retains records according to topic policies such as:

retention.ms

retention.bytes

cleanup.policy

This allows:

- replaying old events;
 - adding a new consumer later;
 - rebuilding derived data;
 - audit and recovery use cases.
-

1.20 Commit

A commit stores the consumer group's processed offset.

Suppose a consumer processes:

Offset 0

Offset 1

Offset 2

After committing, Kafka remembers where that consumer group should continue.

Conceptually:

Committed position = next record to read

After processing offset 2, the committed offset is normally 3.

If the consumer restarts, it can resume from that committed position.

1.21 Consumer Group Coordinator

A broker acts as the coordinator for a consumer group.

It manages:

- group membership;
- consumer heartbeats;
- partition assignment;
- rebalancing;
- committed offsets.

1.22 Controller and KRaft

Kafka uses a controller quorum to manage cluster metadata, broker registration, partition leadership and administrative changes.

Modern Kafka runs using **KRaft**, Kafka's built-in metadata quorum. Important broker-side KRaft settings include:

node.id

process.roles

controller.quorum.bootstrap.servers

log.dirs

These are broker configuration properties, not Spring Boot producer or consumer properties.

2. Complete Producer Flow

Consider the following code in Order Service:

@Service

```
public class OrderEventProducer {
```

```
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;
```

```
    private final String topicName;
```

```
    public OrderEventProducer(
```

```
        KafkaTemplate<String, OrderEvent> kafkaTemplate,
```

```
        @Value("${app.kafka.topic.order-created}") String topicName) {
```

```
        this.kafkaTemplate = kafkaTemplate;
```

```
        this.topicName = topicName;
```

```
    }
```

```
public void publishOrderCreatedEvent(OrderEvent event) {  
  
    kafkaTemplate.send(  
        topicName,  
        event.getOrderNumber(),  
        event  
    );  
}  
}
```

Step 1: Business operation happens

The client places an order:

POST /api/v1/orders

Order Service validates:

Customer exists

Product exists

Requested quantity is valid

Price is available

It saves the order:

Order status = CREATED

Step 2: Producer creates an event

```
OrderEvent event = new OrderEvent();
```

```
event.setOrderId(savedOrder.getId());
```

```
event.setOrderNumber(savedOrder.getOrderNumber());
```

```
event.setCustomerId(savedOrder.getCustomerId());
```

```
event.setProductId(savedOrder.getProductId());
```

```
event.setQuantity(savedOrder.getQuantity());
```

```
event.setTotalAmount(savedOrder.getTotalAmount());
```

The event is a transport representation of the business fact:

“An order has been created.”

Step 3: `KafkaTemplate.send()` is called

```
kafkaTemplate.send(  
    topicName,  
    event.getOrderNumber(),  
    event  
);
```

`KafkaTemplate` delegates the work to the Kafka producer.

The send operation is asynchronous and returns a future representing the eventual send result.

Example:

```
kafkaTemplate  
    .send(topicName, event.getOrderNumber(), event)  
    .whenComplete((result, exception) -> {  
  
        if (exception == null) {  
            System.out.println(  
                "Published to partition: "  
                + result.getRecordMetadata().partition()  
                + ", offset: "  
                + result.getRecordMetadata().offset()  
            );  
        } else {  
            System.err.println(  
                "Publishing failed: " + exception.getMessage()  
            );  
        }  
    });
```

Step 4: Key is serialized

ORD-1001

|

| StringSerializer

v

byte[]

Configured through:

spring.kafka.producer.key-serializer=\

org.apache.kafka.common.serialization.StringSerializer

Step 5: Value is serialized

OrderEvent

|

| JsonSerializer

v

JSON byte[]

Configured through:

spring.kafka.producer.value-serializer=\

org.springframework.kafka.support.serializer.JsonSerializer

Step 6: Partition is selected

Kafka producer determines the destination partition.

The decision can be based on:

1. explicitly supplied partition;
2. record key;
3. producer partitioning strategy when no key is supplied.

With a key:

kafkaTemplate.send(topic, orderNumber, orderEvent);

events with the same order number are normally routed consistently to the same partition.

Without a key:

kafkaTemplate.send(topic, orderEvent);

the producer distributes records according to its configured partitioning logic.

Step 7: Record enters the producer accumulator

The producer normally does not immediately create a separate network request for every record.

It temporarily accumulates records into batches by topic-partition.

Producer accumulator

order-created-0 → Batch A

order-created-1 → Batch B

order-created-2 → Batch C

Batching improves throughput.

Two important properties are:

`spring.kafka.producer.properties.batch.size=16384`

`spring.kafka.producer.properties.linger.ms=5`

Step 8: Producer obtains metadata

The producer uses the bootstrap servers to discover:

- available brokers;
- topics;
- partitions;
- partition leaders.

The configured servers are only the initial contact points; the client learns the complete cluster topology after connecting.

Step 9: Batch is sent to the partition leader

Example:

order-created, Partition 1

Leader = Broker 2

The producer sends the record batch to Broker 2.

Step 10: Broker appends the record

Kafka appends the record to the end of the partition log.

Partition 1

Offset 100 → previous event

Offset 101 → OrderEvent ORD-1001

Kafka assigns the offset.

The producer does not choose the offset.

Step 11: Replicas copy the record

If:

replication factor = 3

the leader's follower replicas copy the record.

Broker 2 → Leader

Broker 1 → Replica

Broker 3 → Replica

Step 12: Broker sends acknowledgement

The producer's acks configuration decides when the send is considered successful.

`spring.kafka.producer.acks=all`

With all, the leader waits for the required in-sync replicas before acknowledging the write.

Step 13: Producer receives metadata

After success, the producer receives:

Topic

Partition

Offset

Timestamp

Example:

Topic = order-created

Partition = 1

Offset = 101

4. Complete Consumer Flow

Suppose Inventory Service contains:

@Component

```
public class OrderEventConsumer {

    @KafkaListener(
        topics = "${app.kafka.topic.order-created}",
        groupId = "${spring.kafka.consumer.group-id}"
    )
    public void consume(OrderEvent event) {

        System.out.println(
            "Received order: " + event.getOrderNumber()
        );

        // Reduce available inventory.
    }
}
```

Step 1: Spring Boot starts the listener container

Spring Boot reads:

spring.kafka.bootstrap-servers=localhost:9092

spring.kafka.consumer.group-id=inventory-group

It configures:

- consumer factory;
- Kafka consumer;
- listener container;
- serializers/deserializers;
- consumer-group membership.

Step 2: Consumer connects to Kafka

The consumer uses:

spring.kafka.bootstrap-servers=localhost:9092

to contact the Kafka cluster.

It discovers:

- brokers;
 - topic metadata;
 - partition leaders;
 - consumer-group coordinator.
-

Step 3: Consumer joins the group

`spring.kafka.consumer.group-id=inventory-group`

Kafka registers the consumer as a member of:

`inventory-group`

Step 4: Kafka assigns partitions

Suppose the topic has three partitions and one Inventory Service instance:

Consumer 1

├─ Partition 0

├─ Partition 1

└─ Partition 2

With three Inventory Service instances:

Consumer 1 → Partition 0

Consumer 2 → Partition 1

Consumer 3 → Partition 2

If there are four consumers but only three partitions:

Consumer 1 → Partition 0

Consumer 2 → Partition 1

Consumer 3 → Partition 2

Consumer 4 → Idle

Within a traditional consumer group, useful parallelism is limited by the number of partitions.

Step 5: Consumer determines the starting offset

Kafka first checks whether the consumer group has a committed offset.

Committed offset exists

The consumer continues from that position.

No committed offset exists

Kafka uses:

`spring.kafka.consumer.auto-offset-reset=earliest`

or:

`spring.kafka.consumer.auto-offset-reset=latest`

Step 6: Consumer polls Kafka

Internally, Kafka consumers use a poll loop:

`consumer.poll(...)`

Spring Kafka's listener container manages this poll loop for you.

Conceptually:

Poll Kafka

|

v

Receive ConsumerRecords

|

v

Deserialize records

|

v

Invoke @KafkaListener

|

v

Commit offsets

|

v

Poll again

A Spring @KafkaListener can receive individual records or batches, depending on listener configuration.

Step 7: Kafka returns records

Kafka returns records from the assigned partitions.

Each record includes:

topic

partition

offset

timestamp

key

value

headers

You can inspect them:

```
@KafkaListener(topics = "${app.kafka.topic.order-created}")
public void consume(
    OrderEvent event,
    @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
    @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
    @Header(KafkaHeaders.OFFSET) long offset) {

    System.out.println("Topic: " + topic);
    System.out.println("Partition: " + partition);
    System.out.println("Offset: " + offset);
    System.out.println("Event: " + event);
}
```

Step 8: Key is deserialized

byte[] → StringDeserializer → String

Configured using:

spring.kafka.consumer.key-deserializer=\

org.apache.kafka.common.serialization.StringDeserializer

Step 9: Value is deserialized

JSON byte[] → JsonDeserializer → OrderEvent

Configured using:

```
spring.kafka.consumer.value-deserializer=\norg.springframework.kafka.support.serializer.JsonDeserializer
```

Step 10: Listener method runs

```
public void consume(OrderEvent event)
```

Inventory Service may then:

1. find inventory using productId;
 2. check available quantity;
 3. deduct ordered quantity;
 4. save inventory;
 5. optionally publish an InventoryUpdatedEvent.
-

Step 11: Offset is committed

After successful processing, the consumer's progress is committed according to the configured acknowledgement mode and consumer settings.

The stored offset belongs to:

Consumer group + Topic + Partition

Example:

inventory-group

order-created

partition 1

committed offset 102

Notification Service has a different offset:

notification-group

order-created

partition 1

committed offset 97

Producer and Consumer Property Matching

These settings must be compatible:

Producer	Consumer	Purpose
StringSerializer	StringDeserializer	Message key
JsonSerializer	JsonDeserializer	Message value
order-created	order-created	Topic

Producer event JSON structure Consumer event structure Data mapping

Example:

Producer value serializer:

OrderEvent → JSON bytes

Consumer value deserializer:

JSON bytes → OrderEvent

The producer and consumer do not have to share the exact same Java package, but their JSON field names and compatible data types should align.

Why Kafka Is Not the Same as a REST Call

REST communication

Order Service

|

| waits for response

v

Inventory Service

Characteristics:

Synchronous

Direct dependency

Receiver must generally be available

Immediate response

Kafka communication

Order Service

|

| publishes event

v

Kafka

|

v

Inventory Service

Characteristics:

Asynchronous

Loosely coupled

Kafka retains records

Consumer can process later

Multiple consumer groups can react independently

Complete Order Workflow

1. Client sends POST /orders.

2. Order Service validates customer and product.

3. Order Service creates the order.

4. Order Service creates OrderEvent.

5. KafkaTemplate.send() is called.

6. StringSerializer serializes the event key.
7. JsonSerializer serializes the OrderEvent.
8. Producer selects a topic partition.
9. Record joins a producer batch.
10. Producer discovers the partition leader.
11. Producer sends the batch to that broker.
12. Broker appends the record to the partition log.
13. Followers replicate the record.
14. Broker acknowledges the producer.
15. Inventory consumer polls Kafka.
16. Kafka returns the record from its assigned partition.
17. StringDeserializer reconstructs the key.
18. JsonDeserializer reconstructs OrderEvent.
19. @KafkaListener receives the event.
20. Inventory Service reduces stock.

21. Consumer offset is committed.

22. Notification Service, using a different group, independently consumes the same event and sends a notification.

15. Most Important Rules to Remember

Topic

Stores a category of related events.

Partition

Provides ordering and parallelism.

Offset

Position of a record inside one partition.

Producer

Publishes events.

Consumer

Reads and processes events.

Broker

Stores events and serves producer/consumer requests.

Cluster

Group of Kafka brokers.

Consumer group

Coordinates consumers and maintains offsets.

Same group ID

Consumers share the work.

Different group IDs

Each group receives its own copy of the event stream.

Serializer

Java object to byte[].

Deserializer

byte[] to Java object.

Record key

Influences partition selection and ordering.

Replication

Maintains additional partition copies for fault tolerance.

Commit

Records the consumer group's progress.

Rebalancing

Redistributes partitions when group membership changes.

Retention

Controls how long Kafka stores records.

DLT

Stores records that could not be processed successfully.

The simplest mental model is:

Producer

|

| serialize

v

Topic → Partition → Offset

|

| deserialize

v

Consumer Group

|

v

Business Processing

|

v

Offset Commit